

Designing Algorithms to Solve Problems



Strathmore
UNIVERSITY



Learning Outcomes

By the end of this tutorial, you should be able to:

- i. Design algorithms using pseudocode and flowcharts.
- ii. Translate simple algorithms into Python programs.
- iii. Apply algorithmic thinking to solve computational problems systematically.

Introduction

- Programming is the process of **translating a solution** into a **language** that a **computer can execute**. Before writing code, a programmer should first design the solution.
- The typical workflow is:
 - Step 1: Understand the Problem
 - Step 2: Break down the problem into smaller bit (Algorithm)
 - Step 3: Write the Pseudocode
 - Step 4: Draw the Flowchart
 - Step 5: Translate into Code (C++, Python)
- **Coding should be the last step, not the first.**

Step 1: Understand the Problem

- The first step is to identify:
 - What is the problem?
 - What inputs are required?
 - What processing is needed?
 - What outputs are expected?

Example

- **Problem-** Calculate the area of a rectangle.
- **Inputs:** Length, Width
- **Processing:** $\text{Area} = \text{Length} \times \text{Width}$
- **Output:** Area

Class Exercise

- A machine learning model can only be deployed if its validation accuracy is at least 90%. Develop a Python program that captures a model name and validation accuracy. If the accuracy is 90% or higher, display "Deploy Model". Otherwise, display "Further Improvement Required".
- **Identify the inputs, processing and outputs required**

Class Exercise 2

- Write a Python program that repeatedly asks the user to guess the secret number. The program should stop when the user enters the correct number. Assume the secret number is 2028
- **Identify the inputs, processing and outputs required**

Step 2: Design the Algorithm

- An algorithm is a **finite sequence** of well-defined, **logical steps** used to **solve a problem** using computers
- Example Algorithm
 - Start
 - Identify inputs
 - Read Length
 - Read Width
 - Determine the processing
 - $\text{Area} = \text{Length} \times \text{Width}$
 - Identify the output
 - Display Area
 - Stop

Notice that this is language-independent. There is no Python syntax.

Step 2: Design the Algorithm

- A good algorithm should have the following characteristics:
 - i. Clearly defined inputs
 - ii. Clearly defined outputs
 - iii. Unambiguous instructions
 - iv. Finite number of steps
 - v. Produces the correct result

Step 3: Write the Pseudocode

- Pseudocode is an informal description of the algorithm using **structured English**.
- Pseudocode provides a structured, **human-readable description** of the algorithm and serves as a bridge between the algorithm and the actual Python program.
 - BEGIN
 - INPUT length
 - INPUT width
 - $\text{area} \leftarrow \text{length} \times \text{width}$
 - OUTPUT area
 - END

Common Pseudocode Keywords

Keyword	Meaning
BEGIN	Start of algorithm
END	End of algorithm
INPUT	Receive data from the user
OUTPUT	Display information
SET	Assign a value
IF	Decision making
THEN	Action after a condition is true
ELSE	Alternative action
END IF	End of decision
WHILE	Repeat while condition is true
FOR	Repeat a fixed number of times
REPEAT	Begin repetition
UNTIL	End repetition when condition is met

Characteristics of good Pseudocode

1. Uses simple English.
2. Clearly identifies inputs and outputs.
3. Includes decision and repetition where needed.
4. Avoids programming-language syntax.
5. Is easy to convert into code.

Benefits of using Pseudocodes ([Video Tutorial](#))

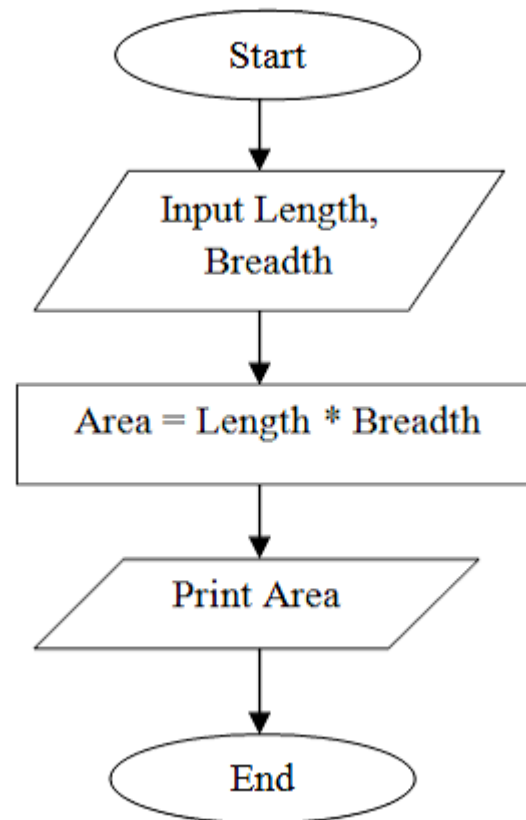
Step 4: Draw the Flowchart

- A flowchart is a diagram that **uses standardized symbols** connected by arrows to represent the logical sequence of operations in an algorithm.
- It provides a **graphical representation** of the algorithm, making it **easier to visualize** the sequence of operations, decisions, and program flow before implementation.
- Instead of describing the solution using words, a flowchart shows the entire process visually.

Standard Flowchart Symbols

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Example Flowchart



Step 5: Translate into Python

```
length = float(input("Enter length: "))  
width = float(input("Enter width: "))  
area = length * width  
print("Area =", area)
```

Step 5: Translate into Python

Observe the relationship:

Pseudocode

INPUT length

INPUT width

area $\leftarrow$ length $\times$ width

OUTPUT area

Python

```
length = float(input())
```

```
width = float(input())
```

```
area = length * width
```

```
print(area)
```

Example 2: Even or Odd Number

- **Problem:** Determine whether a number is even or odd.
- **Inputs:** Number
- **Processing:** Check whether the remainder after dividing by 2 is zero.
- **Output:** Even or Odd

Example 2: Even or Odd Number

- Pseudocode:

```
BEGIN
```

```
INPUT number
```

```
IF number MOD 2 = 0 THEN
```

```
    OUTPUT "Even"
```

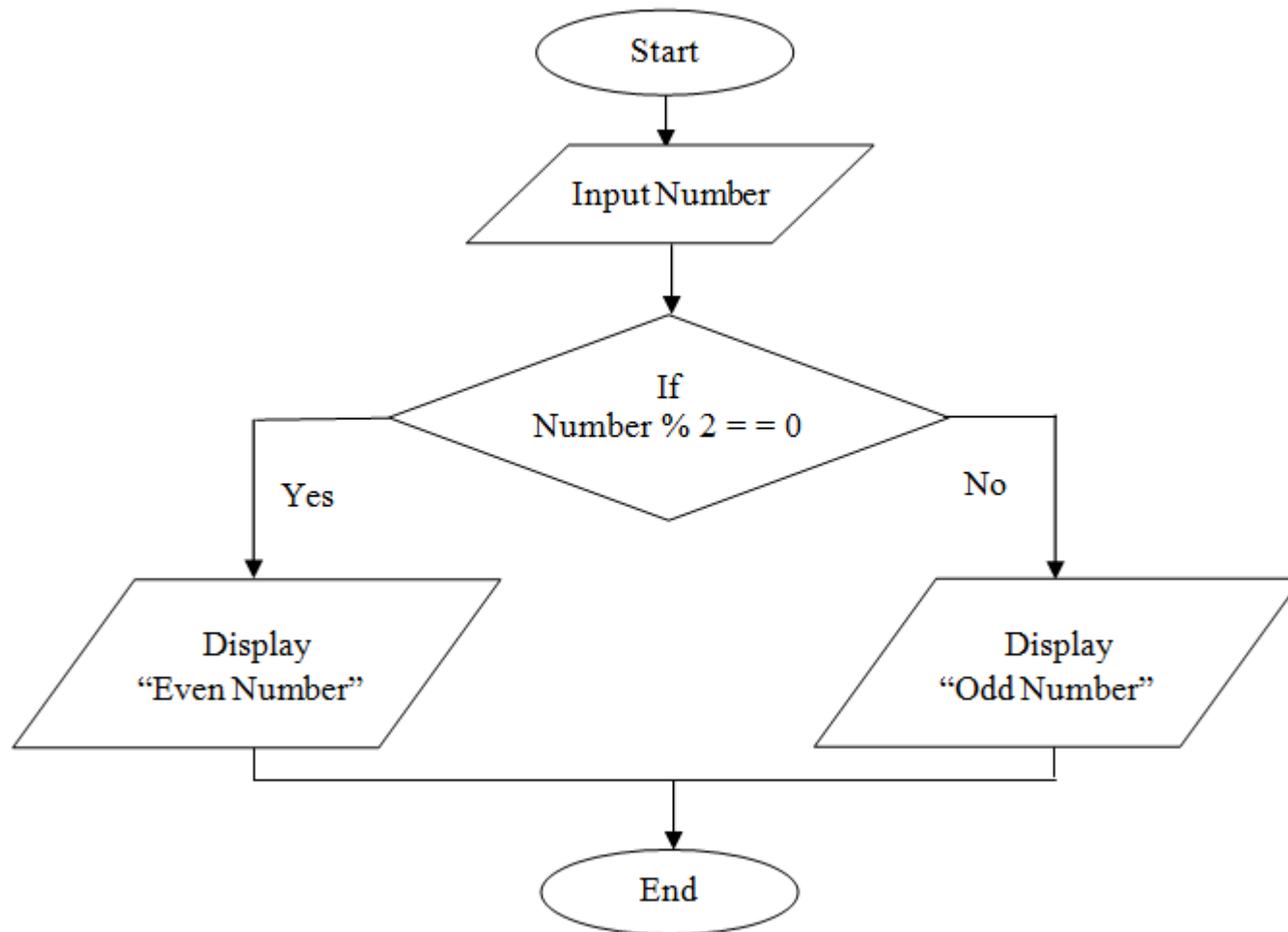
```
ELSE
```

```
    OUTPUT "odd"
```

```
END IF
```

```
END
```

Example 2: Even or Odd Number



Step 5: Translate into Python

```
number = int(input("Enter a number: "))  
if number % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

Class Activity: Employee Payroll Calculation

- ABC Technologies Ltd. pays its employees based on the number of hours worked each week. Employees who work 35 hours or less are paid at their normal hourly rate. To compensate employees who work beyond the standard working hours, the company pays overtime at 1.5 times the normal hourly rate for every hour worked above 35 hours.
- **Tasks**
 - Identify the **Inputs, Processing, and Outputs (IPO)**.
 - Write the pseudocode.
 - Draw the flowchart.
 - Translate the pseudocode into a Python program.

Class Activity: Employee Payroll Calculation

Pseudocode

INPUT hours_worked

INPUT pay_rate

IF hours_worked <= 35 THEN

gross_pay = pay_rate x hours_worked

ELSE

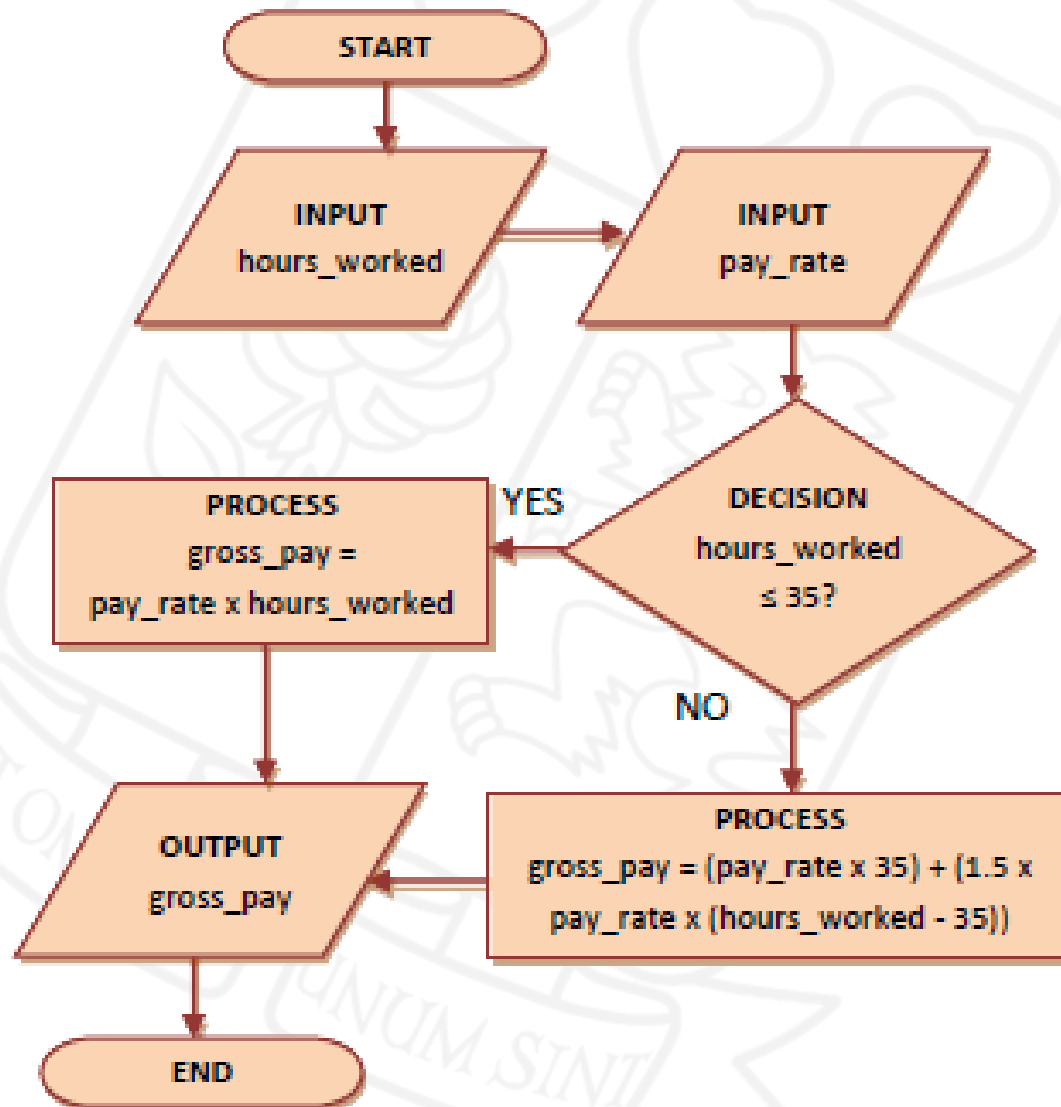
gross_pay = (pay_rate x 35) + (1.5 x pay_rate x
(hours_worked - 35))

ENDIF

OUTPUT gross_pay

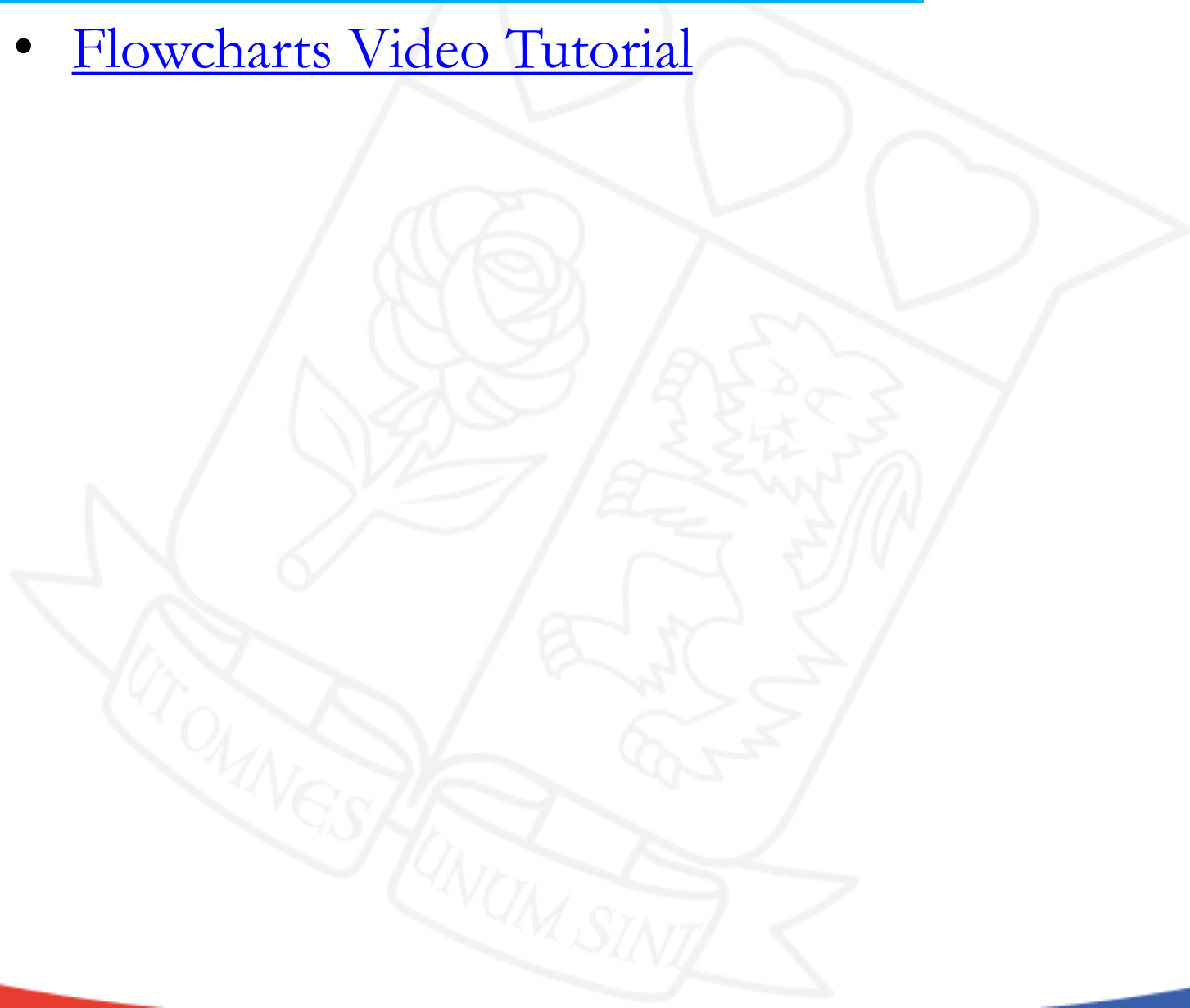
END

Flow Chart



Additional Reference Materials

- [Flowcharts Video Tutorial](#)



Thank you!

Any Questions?